Reconstructing Sprint-Start Jira State for Leakage-Aware Outcome Prediction

Hani AbuSharkh
Higher Colleges of Technology
Al Ain, UAE
habusharkh@hct.ac.ae

Maryam Hanif
United Arab Emirates University
Al Ain, UAE
700052696@uaeu.ac.ae

Rafat Damseh
United Arab Emirates University
Al Ain, UAE
rdamseh@uaeu.ac.ae

Abstract

To predict how a sprint will end, you first have to know what the team knew when it began, and a present-day Jira export does not tell you. For eight open-source TAWOS projects we replay the change log back to the first day of each sprint and rebuild the board as it stood: 2,602 sprints, 1,691 of them with committed work. On a time-ordered split, a scope-only logistic regression ranks spillover among those 1,691 at ROC-AUC 0.885; on the 110 test sprints (94 positive) whose committed scope has fully recorded history it reaches 0.772, so the main estimate is not yet established for data free of static fallbacks. Late closure is harder: gradient boosting reaches 0.751. Dropping unfinished prior sprints from the history features lifts leave-one-project-out late-closure AUC for logistic regression from 0.542 to 0.629. Once a sprint is live, progress features lift spillover AUC from 0.830 to 0.926 at the 25% checkpoint and 0.957 at 50%, while late closure barely changes. Two reproduced leakage mistakes inflate F1 by up to 0.24. What we offer is a change-log reconstruction and a leakage-aware way of evaluating on it, not a deployable risk tool: results vary across projects, the prediction intervals undercover large sprints, and model-based what-if tables say nothing about cause.

Index Terms— *Scrum, sprint analytics, Jira, machine learning, explainability, Shapley values, data leakage*

1 Introduction

A sprint misses its boundary in two ways that Jira records by itself. It spills over when an issue committed at the start is still unresolved at the planned end, and it closes late when Jira stamps its completion more than a day past that end. Neither is a judgment on the team. We ask whether either can be ranked from what was known on the first day of the sprint, and whether watching the sprint unfold adds anything. To keep hindsight out, every feature is rebuilt only from events timestamped at or before the point of prediction.

That line is harder to hold than it sounds, because a present-day Jira export is no snapshot of planning day. Issues move between sprints, estimates get revised, and several sprints are usually open at once, so the current issue-to-sprint link cannot tell work committed at planning from work added later. We replay the TAWOS change log back to each sprint's start and record, per project, where a missing history forces a static fallback. A second leak is easier to miss. In 71% of our sprints the previous sprint had not reached its planned end when the next began, so any summary of it can carry an outcome nobody could have known yet. We measure what excluding those unfinished sprints costs.

What we offer is a data and evaluation contribution, not a deployable risk tool. First, a sprint-start reconstruction from the change log, with an account of how much of it had to fall back on static values. Second, a leakage-aware evaluation of planning-time spillover and late-closure classifiers: per-project time splits, leave-one-project-out (LOPO) testing, project-cluster bootstrap intervals, and explicit checks for zero-scope sprints, history provenance and two reproduced leakage mechanisms. Third, under the same protocol, in-sprint warnings at 25%, 50%, 75% and 100% of the planned duration set against the planning-time models, plus numeric targets and exact Shapley explanations over feature groups. A sprint advisor packages the fitted models and is descriptive only. Fig. 1 lays out the pipeline and Table 2 pins each headline result to its model, split, population and comparator. All of it rests on eight open-source projects, so we read the numbers as evidence about those projects, not as a general benchmark.

2 Related Work

Machine learning on Agile data has mostly chased effort. Choetkiertikul et al. [1] estimated story points from issue text with deep learning; Porru et al. [2] did the same from issue report fields. When Tawosi et al. [3] replicated that work on TAWOS, they found simple baselines kept pace once the feature set was fixed—the pattern we meet again below, where a three-variable scope model holds its own against tree ensembles on spillover. Sprint2Vec [4] instead learns sprint embeddings from issue-level data to regress on how much is delivered and how good it is, while Hearty et al. [5] forecast velocity with a dynamic Bayesian

Table 1: The nearest prior work and what this study adds.

Study	Unit	Target	Sprint-start state
[1]–[3]	Issue	Story points	Not applicable
[4]	Sprint	Delivery and quality	Not reported
[5]	Iteration	Velocity	Not reported
[7]	Sprint	Performance class	In-sprint burndown
[8]	Sprint	Deliverability	Not reported
This work	Sprint	Spillover, late closure	Rebuilt (change log)



Figure 1: The three-stage pipeline. Every stage uses the same reconstructed cohort, the same per-project time split and the same LOPO folds.

network. All predict effort or volume delivered. Of the studies cited here, none targets spillover or late closure as such, and none reports rebuilding the sprint-start state from the change history. Our search was not systematic, so this describes these studies, not the whole field. Table 1 puts them side by side.

Two sprint-level classification studies prompted our leakage checks. Pérez Castillo et al. [7] classified sprint performance from burndown features visible only while the sprint runs, which makes the task retrospective rather than a forecast. Obike et al. [8] reported near-perfect deliverability AUC after running SMOTE [15], but without describing a train-test split taken first. We did not read either implementation and make no claim about their published numbers; instead we reproduce both mechanisms on our own data, to show what each would have cost had it been made (Section IV-G).

For the explanations we take Shapley values from Lundberg and Lee [9], but compute them exactly, over feature groups rather than approximately over single features—Jiarpakdee et al. [11] having shown how correlated metrics distort a per-feature reading; Molnar [10] covers the general case. The nearest relative to our in-sprint setup is outcome-oriented predictive process monitoring [12] and its alarm-based extensions [13], in which a running case is scored at a sequence of prefixes. Among the work we reviewed, we did not find it applied to Scrum sprints with a reconstructed state.

3 Data and Methods

3.1 Dataset and Sprint-Start Reconstruction

Our data comes from the Sprint, Issue and Change_Log tables of TAWOS [6]. We keep the eight open-source Jira projects with at least 30 sprint records apiece—Apache Mesos, Appcelerator Studio, Atlassian Confluence Cloud and Server, LSST Data Management, Mule, Titanium SDK and Titanium Mobile. The export carries 2,657 sprint rows, 145,309 issues and 88,364 change events on the Sprint and Story Points fields. A handful of sprints have to go: the 44 still marked FUTURE or ACTIVE, whose outcome nobody has seen yet, and three CLOSED rows that finish before they begin. Since the history features need a predecessor, we also drop each project's first eligible sprint as a cold start. That leaves 2,602 modelled sprints—911 (35.0%) committing nothing, and 1,691 carrying real scope.

We rebuild the committed scope rather than read it off the final export. Starting from the latest recorded state, the code walks each issue's Sprint and Story Points changes back to the sprint-start timestamp; an issue counts as committed only if, at that moment, it existed, belonged to the sprint, and had not yet been resolved. That gives 20,603 issue-sprint commitments. When an issue has no recorded change for a field, we fall back on its static value and log it—7.2% of commitments lean on the static sprint link and 45.1% on static story points, and 73.5% of issues carry no story points at all and count as zero. Coverage is far from even, from 54.1% of commitments with observed sprint history in Confluence Cloud up to 99.1% in Confluence Server, which is why Section IV-B re-runs the analysis on provenance-restricted subsets.

3.2 Outcomes

Late closure is 1 when Jira's completion timestamp runs more than a day past the planned end, and is defined for all 2,602 sprints. Spillover is 1 when at least one committed issue is still unresolved at the planned end; we score it only on the 1,691 nonempty sprints, since a sprint that commits nothing cannot spill and would be a free correct answer. Three numeric targets sit alongside the labels: issues completed and carry share (the fraction of committed issues left unfinished) on the nonempty sprints, and days late, capped at 30, on all of them. Days late has a long tail: the median sprint closes 0.21 days late, the 90th percentile is 10 days, the worst is 2,603, and 34.5% close early. Table 2 gives each task's population and denominator beside its headline result.

Table 2: Study design and headline results. n (positives) is the test denominator of each figure; checkpoint rows count sprints still open. Differences are row model minus comparator, with 95% project-cluster bootstrap intervals; absolute intervals where the artifact reports them. GB: gradient boosting; RF: random forest; LR: logistic regression; scope only: committed issues, story points and sprint length; strict and legacy name the history feature set.

	Task	Split	n (pos.)	Model, features	Result	Difference [95% CI]	Comparator
1	Late closure	Time	786 (246)	GB, strict	AUC .751	+.111 [.028, .258]	Project rate .640
2	Late closure	Time	786 (246)	GB, strict	AUC .751	+.221 [.098, .369]	Scope-only LR .530
3	Late closure	LOPO	2,602 (1,006)	RF, strict	AUC .642	+.110 [.018, .215]	Scope-only LR .532
4	Late closure	LOPO	2,602 (1,006)	RF, legacy	AUC .600	—	Per project .531–.780
5	Spillover, nonempty	Time	511 (482)	LR, scope only	AUC .885 [.818, .922]	+.073 [.007, .117]	GB, strict .812
6	Spillover, nonempty	LOPO	1,691 (1,575)	LR, scope only	AUC .837	−.017 [−.051, .007]	LR, strict .854
7	Spillover, nonempty	LOPO	1,691 (1,575)	RF, legacy	AUC .815	—	Per project .706–.950
8	Spillover, all sprints	Time	786 (453)	RF, legacy	AUC .986	—	Nonempty .852 [.774, .911]
9	Spillover, full history	Time	110 (94)	LR, scope only	AUC .772, SE ≈ .05	—	Row 5
10	Spillover, 25%	Time	507 (478)	GB, strict + progress	AUC .926	+.097 [.054, .163]	No progress .830
11	Spillover, 50%	Time	501 (472)	GB, strict + progress	AUC .957	+.123 [.068, .188]	No progress .835
12	Late closure, 50%	Time	775 (246)	GB, strict + progress	AUC .760	+.008 [−.008, .031]	No progress .751
13	Issues completed	Time	511	GB (abs. error)	MAE 1.96	−.13 [−.31, .00]	Scope × rate 2.10
14	Carry share	Time	511	GB (abs. error)	MAE .201	−.031 [−.050, −.012]	Train median .232
15	Days late	Time	786	GB (abs. error)	MAE 2.48	−.18 [−.34, .03]	Train median 2.66

3.3 Features and the Parallel-Sprint Timing Fix

Every feature is read off the reconstructed sprint-start state. Scope is the committed issue count and story points; sprint length is the planned duration in days; recent history rolls up the project's earlier sprints—mean delivered points and mean completion ratio over the last three, the last carryover, and how many such sprints exist; project identity is one-hot encoded. The strict set adds the number of other sprints open at the start (mean 1.9, as many as 11), team capacity (mean issues completed and committed over recent sprints, and the new scope against that capacity), and past lateness (mean days late and the share of late sprints). What separates the sets is the meaning of "recent". The legacy set, from our first study, summarises the immediately preceding sprints whether or not they had finished. The strict set admits only sprints whose completion falls before the current start. We run both; results use the strict set unless marked legacy.

3.4 Models and Validation

On the labels we line up an always-positive rule, a project base-rate predictor, a scope-only logistic regression (committed issues, story points, sprint length), logistic regression over all features with balanced class weights, a random forest (300 trees, depth 5, minimum leaf 3, balanced weights), and gradient-boosted trees—XGBoost [16] at 300 trees, depth 4 and a 0.05 learning rate first, and scikit-learn [18] histogram gradient boosting at 200 iterations, depth 3 and the same rate later. The numeric targets get ridge regression, a random forest [17], and gradient boosting under squared-error and absolute-error loss, each against train-median, prior-value and scope-times-completion-rate baselines. We ran no hyperparameter search and fixed the 0.5 threshold for precision, recall and F1 before seeing a result. ROC-AUC is our headline measure, since an always-positive rule already posts F1 0.971 on spillover.

For the main test we sort each project's sprints by time and train on the earliest 70%, leaving 1,816 training and 786 test sprints for late closure and 1,180 and 511 for spillover; we also run 50/50, 60/40 and 80/20 cuts. LOPO holds out a whole project and trains on the other seven, with the held-out project's history still drawn from its own past (Section IV-C). For uncertainty we resample the eight projects as whole units 2,000 times and report 95% percentile intervals of the metric or a paired difference, since sprints within a project are hardly independent. The seed is 42 throughout.

3.5 Explanations, Advisor and Checkpoints

We compute the Shapley values exactly, over the seven feature groups, in their interventional form: for a given sprint and each of the 128 group coalitions, the coalition's features are pinned to that sprint's values while the rest are drawn from a background sample of 40 training sprints, and the coalition is scored by the model's mean prediction over that sample. The attributions then sum exactly to the prediction minus the background mean, and for each group we report its share of the mean absolute attribution over 150 random test sprints (100 at the in-sprint checkpoint). The advisor wraps the fitted models: for a planned sprint it returns both probabilities, the expected issues completed with an 80% interval from quantile boosters, the median and 90th-percentile days late, and a what-if table across alternative scopes. A what-if table is only a model's expectation, so we also set the heaviest quartile of scope-relative-to-capacity against the lightest, on realised carry share and lateness, with cluster bootstrap intervals.

For the in-sprint work we rebuild the sprint state at 25%, 50%, 75% and 100% of the planned duration, from events time-stamped at or before each point: committed issues done and remaining, points left, the gap between work finished and time elapsed, the observed and required completion rates, and issues or points added or dropped since the start. A sprint already closed by a checkpoint is dropped from it, its outcome no longer in doubt (98.6%

Table 3: Planning-time results, strict features, time-ordered split (fixed 0.5 threshold).

Task	Model	n	F1	AUC	Brier
Late	Always positive	786	.477	.500	—
Late	Scope-only LR	786	.336	.530	.244
Late	Project base rate	786	.470	.640	.221
Late	Logistic regr.	786	.495	.680	.224
Late	Random forest	786	.502	.715	.207
Late	Gradient boosting	786	.553	.751	.176
Spill.	Always positive	511	.971	.500	—
Spill.	Scope-only LR	511	.696	.885	.230
Spill.	Logistic regr.	511	.842	.827	.161
Spill.	Random forest	511	.880	.840	.115
Spill.	Gradient boosting	511	.974	.812	.049

remain open at 25%, 96% at 75%, 65% at 100%), and we do not run spillover at 100% because the state settles it. As before, each progress-aware model is set against the same algorithm on planning-time features alone and against a learning-free rule—one minus the share done, for spillover; committed issues still open, for late closure—on the same split.

4 Results

4.1 Planning-Time Classification

Table 3 collects the main time-split results under strict features. Every spillover figure here is on nonempty sprints, and Section IV-B adds the stricter provenance check. Late closure is rankable, if only moderately: gradient boosting reaches ROC-AUC 0.751, the random forest 0.715 and logistic regression 0.680, while the scope-only model lingers at 0.530—essentially a coin toss. Gradient boosting is the one model whose paired cluster-bootstrap interval clears the project base rate of 0.640 (Table 2, rows 1 and 2). Spillover turns this on its head. The scope-only model hits 0.885 (cluster interval 0.818 to 0.922) and everything richer trails it: logistic regression 0.827, gradient boosting 0.812 (row 5), and the random forest 0.840, whose paired interval straddles zero. That 0.885 rests in part on static fallbacks—7.2% of commitments for the sprint link, 45.1% for story points—whose present-day values can carry hindsight into a planning-time feature; Section IV-B rescores without them.

Population matters as much. With legacy features the spillover forest scores 0.986 AUC on all 2,602 sprints and 0.852 on the 1,691 nonempty ones. That gap is the 35% of sprints negative by construction—so a study that keeps them looks roughly 0.13 AUC stronger than it is. Nor does it ride on the 70/30 cut: across 50/50 to 80/20 splits the two models stay within 0.669 to 0.711 and 0.865 to 0.887.

4.2 Timing and Provenance

Swapping legacy history features for strict ones lifts late-closure AUC for most models under both protocols, and the lift is biggest where it counts most—across projects. Under LOPO, logistic regression climbs from 0.542 to 0.629 and the random forest from 0.600 to 0.642; on the time split, logistic regression goes 0.636 to 0.680 and gradient boosting 0.726 to 0.751. Spillover barely stirs—about 0.02 either way, inside the noise—since scope is identical in both sets. No headline flips, but the legacy features were less informative than they looked, and have no business being called known at planning time.

Provenance is the bigger caveat, and the main spillover estimate should be read with it. We rerun the scope-only model and the legacy forest on two nested subsets. Require that every committed item have at least one Sprint-field change recorded before the sprint start, so that its membership is observed rather than assumed, and 1,174 sprints remain (356 in the test period, 335 positive): the scope-only model scores 0.875 and the forest 0.850. Require a recorded Story Points change as well, and 358 remain (110 in the test period, 94 positive and 16 negative): 0.772 and 0.652 (row 9). The artifact carries no bootstrap for these subsets, so we give the analytic Hanley–McNeil standard error instead: about 0.05 for the smaller one, an approximate 95% range of 0.67 to 0.88 that overlaps the main estimate's 0.818 to 0.922. With 16 negatives, the drop is as consistent with sampling noise as with fallback dependence, and we do not read 0.77 as a replacement estimate. What the subset establishes is narrower: the 0.885 has not been shown to hold on data free of fallback values, and 0.77 is the only estimate we have for such data. Label thresholds matter far less: the legacy late-closure forest moves only from 0.696 with no grace day to 0.626 with seven, and stricter spillover definitions (two unfinished issues, or a completion ratio under 0.8) keep full-population AUC above 0.97.

4.3 Cross-Project Generalization

LOPO keeps each held-out project's own earlier sprints as history inputs, so it asks whether a rule learned on seven projects carries to an eighth that has a track record. It is not a cold start for a new project. Within that test it is the project, far more than the model, that moves performance. With legacy features the late-closure forest pools to 0.600 but runs from 0.531 on Titanium Mobile—48 late sprints out of 212—up to 0.780 on Appcelerator Studio; the nonempty-sprint spillover forest from 0.706 on Confluence Cloud, where 267 of 464 sprints commit nothing and the rest are hard to tell apart, up to 0.950 on Appcelerator Studio. And under LOPO the plain always-late rule (F1 0.558) beats every learned model on F1: a threshold tuned elsewhere does not travel. Concept drift [14] is part of why: under our split, Titanium SDK's late-closure rate drops from 71% in training to 33% in test, and Apache Mesos's from 38% to 9%.

4.4 What Explains Each Outcome

Fig. 2 lays out the exact Shapley shares for the two planning-time classifiers. For the fitted late-closure model,

Table 4: In-sprint ROC-AUC by checkpoint: gradient boosting with and without progress features, and the learning-free progress rule. Each checkpoint scores only sprints still open at that point, so the planning-only column shifts at 100%.

Task	Model	Split	25%	50%	75%	100%
Spill.	Planning only	Time	.830	.835	.827	—
Spill.	With progress	Time	.926	.957	.953	—
Spill.	Progress rule	Time	.667	.767	.802	—
Spill.	Planning only	LOPO	.801	.815	.804	—
Spill.	With progress	LOPO	.860	.897	.927	—
Spill.	Progress rule	LOPO	.610	.715	.827	—
Late	Planning only	Time	.751	.751	.751	.717
Late	With progress	Time	.754	.760	.749	.704
Late	Progress rule	Time	.479	.483	.486	.484
Late	Planning only	LOPO	.645	.652	.647	.672
Late	With progress	LOPO	.658	.667	.664	.657
Late	Progress rule	LOPO	.518	.517	.521	.510

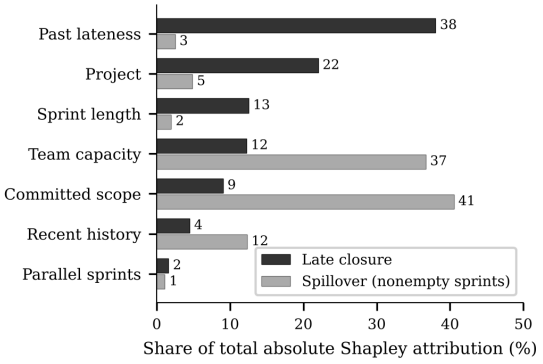


Figure 2: Share of mean absolute exact Shapley attribution by feature group for the two random-forest classifiers (strict features), over 150 time-split test sprints.

past lateness takes 38% of the attribution and project identity 22%, with sprint length and capacity at 13% and 12%, and committed scope a mere 9%. For spillover, committed scope takes 41% and capacity 37%, the model weighing how much was taken on against how much the team usually clears. The days-late model echoes late closure (project 26%, past lateness 26%, sprint length 22%), and the issues-completed model is 71% scope. A group ablation on the legacy forest agrees: pull scope out and nonempty spillover AUC collapses from 0.852 to 0.652, while late closure barely moves, 0.705 to 0.690. The legacy forests' reliability bins warn against reading their scores as probabilities—the late-closure forest's 0.7–0.8 bin runs a 35% late rate, the spillover forest's 0.3–0.4 bin a 90% spillover rate—so we use the scores only as rankings.

4.5 Numeric Targets and the Advisor

Issues completed is mostly arithmetic. Multiply committed issues by the team's recent completion rate and the mean absolute error is 2.10 issues (R^2 0.42); the best learned model, a booster trained on absolute error, manages 1.96 (R^2 0.46), a difference whose interval grazes zero (row 13). Carry share is all but unpredictable (R^2 near zero for every model), though that booster does trim the error from 0.232 to 0.201 (row 14). Days late punishes squared-error models, whose 3.7-to-4.0-day errors are worse than the training median's 2.66; the absolute-error booster reaches 2.48, no real improvement. The advisor's 80% prediction intervals for issues completed cover 84% of outcomes on the time split and 85% across projects, a third tighter than the scope-times-rate baseline (5.4 against 8.4 issues)—but on the time split coverage sags to 70% once a sprint carries 13 or more issues (80% across projects). And the late-closure probability is calibrated on the time split (top quartile predicted 0.66, observed 0.65) but not across projects (bottom quartile 0.16 against 0.30).

The what-if table is not advice. Sprints planned heaviest relative to capacity carry over just 3.2 percentage points more of their scope than the lightest quartile (interval −2.3 to +8.4) and close late 5.6 points more often (−5.2 to +16.4); within projects the load–carry-share correlation is positive in two of eight and load-lateness in five of eight. The fitted late-closure probability is not even monotone in scope—in one LSST case it drops from 0.555 at 8 issues to 0.311 at 38.

4.6 In-Sprint Early Warning

Table 4 shows what progress buys. For spillover it buys a great deal, and early: on the time split gradient boosting moves from 0.830 on planning features to 0.926 at the 25% checkpoint, 0.957 at 50% and 0.953 at 75%—gains of +0.10, +0.12 and +0.13 over the same model without progress, each with an interval clear of zero (0.054 to 0.163 at 25%). Across projects the gains are +0.06, +0.08 and +0.12, again significant for gradient boosting, though at 25% the logistic and forest gains do not clear zero. The learning-free rule reaches 0.667, 0.767 and 0.802, so the model is worth between 0.10 and 0.26 AUC over a bare count of open issues—time pressure, scope and history adding to raw progress. Carry share is humbler: the mean absolute error drops from about 0.20 on planning features to 0.146, 0.112 and 0.070 with progress, but the rule "carry share is one minus the share done so far" already scores

0.165, 0.115 and 0.064, and on the time split the model stops beating the rule from halfway on.

Late closure, by contrast, barely moves. Gradient boosting scores 0.751 on planning features and 0.754, 0.760, 0.749 and 0.704 with progress at the four checkpoints. Of the 24 paired comparisons with the same model without progress, 22 intervals include zero, and the other two gains are under 0.03 AUC (the forest at 25% on the time split, +0.020; gradient boosting at 50% across projects, +0.015). The open-count rule hovers near 0.5. In these data, open-work progress adds little information about whether Jira closes a sprint late. At the planned end 65% of sprints are still open and 51% of those close more than a day late, yet no model beats always predicting late by anything that matters (F1 0.68 versus 0.68). The 50% explanations agree: for spillover the mix shifts to progress so far 34%, time pressure 23%, scope 16% and recent history 15%, while for late closure past lateness still holds 37%, project 17%, and progress only 6%.

4.7 Leakage Case Studies

We reproduce two mistakes on our own data, scoring F1 at the fixed threshold before and after each. Feed in the sprint's own completion ratio—known only once the sprint is over—and spillover F1 jumps from 0.756 to 0.991 and AUC from 0.834 to 1.000, because the feature just restates the label; late closure hardly stirs (+0.010), since finishing the work is not what decides when somebody clicks "complete sprint". Run SMOTE over the whole pool before splitting and late-closure F1 inflates from 0.457 to 0.646, as synthetic copies of test sprints leak into training, while spillover is untouched—at 94% positives there is next to nothing to oversample. Neither mistake inflates results by a fixed amount; the damage depends on the mechanism and the label, so a leakage audit must be run per task.

5 Threats to Validity

Construct. Jira's completion timestamp marks when a sprint was closed, not when its work was done, so late closure is an administrative event. Spillover says only that at least one committed item went unresolved. Neither label measures how well a team performed.

Reconstruction. Replaying the change log beats trusting a static export, but what was never recorded cannot be recovered, and Section IV-A shows how the story-point fallback caps the spillover estimate. The strict history features close the parallel-sprint leak, yet cannot promise that the rebuilt state is what the team actually had in front of it. We read completion from each issue's final resolution date, so an issue reopened and resolved again counts at that later resolution—which can only make early progress look slower.

Evaluation. Eight projects make for wide cluster intervals, and our bootstrap over projects is exploratory, not confirmatory. We ran no decision-utility analysis. Prevalence drives the thresholded metrics, and the drift within projects means any deployed model would need local recalibration. The Shapley values describe the fitted models, not causes.

External. Every project here is open-source. Enterprise teams that run to velocity targets and close sprints on schedule may behave differently.

6 Conclusion

What this study contributes is a protocol more than a tool, and under it reconstructed Jira history does carry signal—real, but specific to each outcome. Spillover is a work outcome: committed scope accounts for most of it at planning time (scope-only AUC 0.885 on nonempty sprints, time split), gradient boosting with progress reaches 0.96 at mid-sprint, and at 94% prevalence the question worth asking is which sprints carry the most unfinished work, not whether any will. Late closure is an administrative-timing outcome: planning-time features reach 0.751 only once history is held to sprints that had genuinely finished, past lateness and project identity take 60% of the forest's attribution, and progress adds nothing. Performance varies widely from project to project, so the numbers themselves should not be expected to carry over. The methodological lessons are about data and evaluation—rebuild the sprint-start state, drop the sprints that commit nothing, honour parallel-sprint timing, and audit for leakage one label at a time, since the same mistake costs 0.24 F1 on one outcome and 0.01 on the other.

What can a team actually do with this? Three narrow things. At planning, the scores rank sprints by risk, so a team knows which ones to watch; they are rankings, not calibrated odds, and any alert threshold has to be set locally for the cost of a false alarm. During the sprint, the 25% and 50% checkpoints flag likely spillover earlier and better than counting open issues. The issue-count intervals are honest but widen for large sprints. We have not shown that acting on a score improves delivery, or that cutting scope causes a lower carry share; that needs a prospective or controlled study.

Every result runs off a fixed seed (42) on a laptop CPU, under Python 3.13 with scikit-learn 1.8, pandas 2.2 and XGBoost 3.1. The artifact at `https://github.com/hani-ziad/SwEng_202610` holds the data export, the reconstruction and modelling code, 50 automated tests, and every result file behind the numbers above. Next is a silent prospective deployment that scores sprints without steering the team.

References

[1] M. Choetkiertikul, H. K. Dam, T. Tran, T. Pham, A. Ghose, and T. Menzies, "A deep learning model for

estimating story points," IEEE Trans. Softw. Eng., vol. 45, no. 7, pp. 637–656, Jul. 2019.

[2] S. Porru, A. Murgia, S. Demeyer, M. Marchesi, and R. Tonelli, "Estimating story points from issue reports," in Proc. 12th Int. Conf. Predictive Models Data Analytics Softw. Eng. (PROMISE), 2016, pp. 1–10.

[3] V. Tawosi, R. Moussa, and F. Sarro, "Agile effort estimation: Have we solved the problem yet? Insights from a replication study," IEEE Trans. Softw. Eng., vol. 49, no. 4, pp. 2677–2697, Apr. 2023.

[4] M. Choetkiertikul, P. Banyongrakkul, C. Ragkhitwetsagul, S. Tuarob, H. K. Dam, and T. Sunetnanta, "Sprint2Vec: A deep characterization of sprints in iterative software development," IEEE Trans. Softw. Eng., vol. 51, no. 1, pp. 220–242, Jan. 2025.

[5] P. Hearty, N. Fenton, D. Marquez, and M. Neil, "Predicting project velocity in XP using a learning dynamic Bayesian network model," IEEE Trans. Softw. Eng., vol. 35, no. 1, pp. 124–137, Jan. 2009.

[6] V. Tawosi, A. Al-Subaihin, R. Moussa, and F. Sarro, "A versatile dataset of agile open source software projects," in Proc. 19th Int. Conf. Mining Softw. Repositories (MSR), 2022, pp. 707–711.

[7] Y. J. Pérez Castillo, S. D. Orantes Jiménez, and P. O. Letelier Torres, "Sprint management in Agile approach: Progress and velocity evaluation applying machine learning," Information, vol. 15, no. 11, Art. no. 726, 2024.

[8] P. G. Obike, V. E. Ekong, and O. U. Obot, "A novel scoring-based Agile sprint deliverability prediction and prioritization using machine learning," Int. J. Comput. Appl., vol. 187, no. 49, pp. 40–53, Oct. 2025.

[9] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems 30 (NIPS 2017), 2017, pp. 4765–4774.

[10] C. Molnar, Interpretable Machine Learning: A Guide for Making Black Box Models Explainable, 2nd ed., 2022. [Online]. Available: `https://christophm.github.io/interpretable-ml-book/`

[11] J. Jiarpakdee, C. Tantithamthavorn, and A. E. Hassan, "The impact of correlated metrics on the interpretation of defect models," IEEE Trans. Softw. Eng., vol. 47, no. 2, pp. 320–331, Feb. 2021.

[12] I. Teinemaa, M. Dumas, M. La Rosa, and F. M. Maggi, "Outcome-oriented predictive process monitoring: Review and benchmark," ACM Trans. Knowl. Discov. Data, vol. 13, no. 2, Art. no. 17, 2019.

[13] S. A. Fahrenkrog-Petersen, N. Tax, I. Teinemaa, M. Dumas, M. de Leoni, F. M. Maggi, and M. Weidlich, "Fire now, fire later: Alarm-based systems for prescriptive process monitoring," Knowl. Inf. Syst., vol. 64, no. 2, pp. 559–587, 2022.

[14] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," ACM Comput. Surv., vol. 46, no. 4, Art. no. 44, 2014.

[15] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority oversampling technique," J. Artif. Intell. Res., vol. 16, pp. 321–357, 2002.

[16] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, 2016, pp. 785–794.

[17] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.

[18] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.